

N100 Financial Intelligence Platform

Technical Documentation

Architecture, data model and design reference

92 companies | 14 tables | 30+ KPIs | 20 API endpoints
311 tests | 23 deliverables | 20 of 20 acceptance gates

**Reviewers: chapter 0 explains how the six form uploads fit together
and how to get the project running.**

Version 1.0 • 2026-07-28

Raj Sarania, Data Analyst

Summer Internship Programme, Bluestock Fintech

0. About this submission

Read this first. It explains how the six uploads fit together.

This project was submitted through a form with six slots: three source code archives, a database archive and two PDF documents. **The complete project is the union of all six uploads.** No single upload is the whole thing.

Form slot	What it contains
1. Source Code (Frontend)	The Streamlit dashboard: entry point, all 8 screens, cached data loader. Also carries the 92 radar chart PNGs used by the peer views
2. Source Code (Backend)	The analytics engine: ETL, ratio engine, screener, NLP rule engine, clustering, statistics and report generators. Also carries the generated Excel workbooks, the 91 company tearsheets, 11 sector reports and the portfolio summary
3. Source Code (RestAPI)	The FastAPI application: 20 endpoints across 7 routers, the OpenAPI 3.1 specification and the full test report
4. Database SQL File	The DDL schema, the populated SQLite database, all 12 source Excel workbooks so the database can be rebuilt from scratch, exploratory queries and the load and validation audit trail
5. Technical Documentation	This document: architecture, data model, formulas, algorithm designs, API contract, test strategy and known limitations
6. Daily Reports	The project progress record: 9 day-level logs and 6 sprint retrospectives, with a coverage map stating which days are recorded at which level

Why the segmentation looks unusual

The form is shaped for a conventional web application with a separate frontend, backend and API. This project is a Python analytics platform, so the mapping is: the Streamlit dashboard is the frontend, the ETL and analytics engine is the backend, and FastAPI is the API. The form also has no slot for the 23 tracked deliverables, so each one is attached to the segment that produces or displays it. Every archive contains a MANIFEST.txt naming the other five uploads.

Getting it running in five minutes

Extract all four archives into one directory. Each archive has a single top-level folder; merge their contents so that `src/`, `data/`, `config/` and `tests/` sit side by side. Then:

```
pip install -r requirements.txt

# the database ships populated, so start here:
python -m pytest -q # 311 tests
streamlit run src/dashboard/app.py # dashboard on :8501
uvicorn src.api.main:app --port 8000 # API on :8000, /docs

# or rebuild everything from the source Excel workbooks:
python -m src.etl.sqlite_loader # 12 tables, 12,892 rows
python -m src.etl.ratio_engine # 1,184 KPI rows
python -m src.screener.run_screener # screener + peer ranks

# a Makefile wraps every step: make help
```

This sequence was verified by extracting the four archives into an empty directory, deleting the shipped database and rebuilding from the Excel workbooks: 311 tests pass and all 20 acceptance gates hold. Python 3.12 or newer is required.

Where to look for what

Question	Answer
Is it complete and correct?	docs/acceptance_checklist.pdf, 20 gates with evidence
How do I use the platform?	docs/analyst_guide.pdf, an 11 page user manual
How does it work internally?	This document, chapters 1 to 8
What was built when, and why?	Upload 6, and docs/Sprint1..6_retrospective.md
What does the test suite cover?	reports/pytest_report.html
Why does a ratio look odd?	output/ratio_edge_cases.log, every anomaly categorised

Question	Answer
How fast is it?	output/perf_notes.md

Two things stated plainly up front

First: the market_cap and stock_prices datasets supplied with this project are SIMULATED, so every P/E, P/B, EV/EBITDA, dividend yield and market cap figure is illustrative rather than real market data. Second: the supplied companies.xlsx is truncated at 92 rows, and several other source defects were found during the build. All of them are documented in chapter 7 rather than worked around silently.

1. System architecture

A four-layer batch analytics platform over a single SQLite file.

The platform is deliberately a batch pipeline with read-only presentation on top, not a live service. Excel files are the system of record; SQLite is the working store; every analytical figure is derived, never hand-entered. Nothing writes back to the source data.

Layer model

```
SOURCE 12 Excel workbooks (raw + supporting)
|
v src/etl/sqlite_loader.py
STORE data/nifty100.db 14 tables
|
v src/etl/ratio_engine.py
DERIVED financial_ratios 1,184 rows, 17 KPI cols
output/capital_allocation.csv
|
v src/analytics/ src/screener/ src/nlp/
ANALYTICS screening universe, composite score,
peer percentiles, clustering, statistics,
valuation flags, rule-based pros and cons
|
+-----+-----+-----+
v v v v
PRESENT Streamlit FastAPI Excel ReportLab
8 screens 20 routes 4 books PDFs
```

Why this shape

- **The expensive work happens once.** Ratio computation, clustering and PDF generation are batch steps whose output is committed. The dashboard and API read that output rather than recomputing it.
- **One analytics layer, three consumers.** The dashboard screener, the API screener endpoint and the Excel export all call the same ScreenerEngine. They cannot disagree about what a filter means, and an acceptance gate asserts the API and the workbook return identical company sets.
- **Derived values are never stored twice.** ROCE and the sector-relative score are recomputed on read rather than persisted, because the Sprint 2 schema has no column for them and a second copy would drift.
- **SQLite is sufficient.** The largest table is 5,520 rows. Query time is measured in tens of microseconds. A server database would add operational cost for no gain.

Package layout

Package	Responsibility
src/etl/	Excel to SQLite loading, validation, ratio engine, edge case audit, index creation
src/analytics/	Pure calculation: ratios, cash flow KPIs, CAGR, period parsing, peer ranks, valuation, clustering, statistics
src/screener/	Screening universe assembly, filter engine, composite score, Excel export
src/nlp/	Analysis text parsing, per-company feature frames, the 24-rule pros and cons engine
src/reports/	ReportLab PDFs, Excel workbooks, radar charts, acceptance gates, deliverable archive
src/dashboard/	Streamlit app, 8 screens, cached loader
src/api/	FastAPI app, dependencies, 7 routers
tests/	311 tests across etl, kpi, dq, analytics, screener, nlp, reports, dashboard, api

The dependency direction is strictly one way: analytics never imports from presentation, and pure calculation modules never import a database connection. That is what allows the ratio functions to be unit tested with plain numbers.

2. Data model

Live schema read from data/nifty100.db at generation time.

Table	Rows	Columns
analysis	20	id, company_id, compounded_sales_growth, compounded_profit_growth, stock_price_cagr, roe
balancesheet	1,312	id, company_id, year, equity_capital, reserves, borrowings, other_liabilities ...
cashflow	1,187	id, company_id, year, operating_activity, investing_activity, financing_activity, net_cash_flow
companies	92	id, company_logo, company_name, chart_link, about_company, website, nse_profile ...
documents	1,585	id, company_id, Year, Annual_Report
financial_ratios	1,184	id, company_id, year, net_profit_margin_pct, operating_profit_margin_pct, return_on_equity_pct, debt_to_equity ...
market_cap	552	id, company_id, year, market_cap_crore, enterprise_value_crore, pe_ratio, pb_ratio ...
peer_groups	56	id, peer_group_name, company_id, is_benchmark
peer_percentiles	550	id, company_id, peer_group_name, metric, value, percentile_rank, year
profitandloss	1,276	id, company_id, year, sales, expenses, operating_profit, opm_percentage ...
prosandcons	16	id, company_id, pros, cons
sectors	92	id, company_id, broad_sector, sub_sector, index_weight_pct, market_cap_category
stock_prices	5,520	id, company_id, date, open_price, high_price, low_price, close_price ...

Keys and joins

- **company_id** is the join key everywhere. It is the NSE ticker, normalised to upper case and trimmed at load time.
- **year** is a text period label, not a number. Values look like Mar 2024 or Dec 2023, and companies do not share a financial year end.
- **market_cap.year is an integer** (2019 to 2024), so joining valuation to statements requires mapping the period label to the calendar year it closes in. `src/analytics/periods.py` owns that mapping.
- **financial_ratios** is fully derived. It is deleted and rebuilt by the ratio engine on every run; nothing else writes to it except the composite score update.
- **peer_percentiles** is derived from the universe and rebuilt by the peer engine.

The period problem

The raw workbooks mix clean labels with irregular ones: TTM, Mar 2023 15 and Mar 2016 9m all appear. Ordering periods by string sort would be wrong, and any CAGR window built on a wrong order is silently incorrect.

`periods.py` is the single answer to this. It converts a label to a sortable integer (`calendar_year * 100 + month`), returns -1 for anything unparseable so it can be filtered out, and exposes `latest_rows()` and `deduplicate_company_years()` so every consumer agrees on what "the latest year" means.

Duplicate company-year rows

The source statements contain 119 duplicate company-year rows. PNB carries 60 rows for 12 distinct years. Sprint 2 decided to preserve them rather than silently modify source data, so every analytics consumer must de-duplicate before counting. A screener built directly on `financial_ratios` would report the same company several times.

3. KPI definitions

Every formula the ratio engine implements, with its edge case behaviour.

KPI	Formula	Returns None when
Net profit margin	$\text{net_profit} / \text{sales} \times 100$	sales = 0
Operating margin	$\text{operating_profit} / \text{sales} \times 100$	sales = 0
ROE	$\text{net_profit} / (\text{equity} + \text{reserves}) \times 100$	equity + reserves <= 0
ROCE	$(\text{operating_profit} + \text{other_income}) / (\text{equity} + \text{reserves} + \text{borrowings}) \times 100$	capital employed <= 0, or any input missing
Debt to equity	$\text{borrowings} / (\text{equity} + \text{reserves})$	equity + reserves <= 0. Returns 0 when borrowings = 0
Interest coverage	$(\text{operating_profit} + \text{other_income}) / \text{interest}$	interest = 0, which means Debt Free
Asset turnover	$\text{sales} / \text{total_assets}$	total_assets = 0
Free cash flow	$\text{operating_activity} + \text{investing_activity}$	never; negative is a valid outcome
CFO quality	$\text{operating_activity} / \text{net_profit}$	net_profit = 0
CapEx intensity	$\text{abs}(\text{investing_activity}) / \text{sales} \times 100$	sales = 0
FCF conversion	$\text{free_cash_flow} / \text{operating_profit} \times 100$	operating_profit = 0
CAGR (n years)	$((\text{end} / \text{start}) ^ (1/n) - 1) \times 100$	start = 0, both negative, decline to loss, or turnaround
FCF yield	$\text{free_cash_flow} / \text{market_cap} \times 100$	market cap missing or 0

Label bands

Metric	Bands
CFO quality	above 1.0 High Quality Earnings, 0.5 to 1.0 Moderate, below 0.5 Accrual Risk
CapEx intensity	below 3% Asset Light, 3 to 8% Moderate, above 8% Capital Intensive
FCF conversion	above 60% Efficient, 30 to 60% Moderate, below 30% CapEx Heavy
Interest coverage	null means Debt Free; below 1.5 raises a warning flag
Valuation flag	P/E above 1.5x sector median Caution, below 0.7x Discount, otherwise Fair

Capital allocation classifier

Companies are labelled from the signs of the three cash flow lines. Seven of the eight possible sign combinations are named by the specification; the eighth, (-, +, -), is not, and the two companies that fall there are labelled Unknown Pattern.

CFO	CFI	CFF	Pattern
+	-	-	Reinvestor, or Shareholder Returns when CFO/PAT is above 1.0
+	+	-	Liquidating Assets
+	+	+	Cash Accumulator
+	-	+	Mixed
-	+	+	Distress Signal
-	-	+	Growth Funded by Debt
-	-	-	Pre-Revenue
-	+	-	Not named by the specification

Operating profit is EBITDA by construction

The source P&L reports operating profit before depreciation. This was verified against the identity $\text{operating profit} + \text{other income} - \text{interest} - \text{depreciation} = \text{profit before tax}$, which holds with a median residual of 1.0 across 1,263 rows, against 1,317 for the post-depreciation reading. Note that depreciation IS present in the data; an earlier revision of this document claimed otherwise and was wrong.

4. Screener and scoring design

Filter engine, business rules and the composite quality score.

The screening universe

`src/screener/universe.py` assembles one row per company at its latest financial year, joining `financial_ratios`, `profitandloss`, `balancesheet`, `market_cap`, `sectors` and `companies`, then deriving ROCE, CFO/PAT, the 3-year revenue CAGR, the 5-year FCF CAGR and a debt-declining flag. Everything downstream reads this one frame.

Configuration versus code

Thresholds live in `config/screener_config.yaml` so an analyst can change them without touching Python. Business rules live in code, because they are not numbers.

Rule	Behaviour	Why
Financials D/E exemption	An upper-bound D/E filter ($<$ or $<=$) does not reject Financials. An equality filter such as $D/E == 0$ still applies to them	Leverage of 8 is structurally normal for a lender, but a bank with D/E 8 is genuinely not debt free
Debt Free coverage	A null interest coverage passes any minimum	Null means interest expense is zero, so coverage is infinite, not unknown
Everything else	A missing value fails the filter	A company cannot be asserted to meet a threshold on absent data

Composite quality score

A 0 to 100 score blending four components. Each input metric is winsorised at the 10th and 90th percentile, then min-max scaled across the winsorised range, then weighted.

Component	Weight	Inputs
Profitability	35	ROE 15, ROCE 10, net profit margin 10
Cash quality	30	FCF CAGR 15, CFO/PAT 10, FCF positive flag 5
Growth	20	Revenue CAGR 10, PAT CAGR 10
Leverage	15	D/E score 10, interest coverage score 5

- **Winsorisation is load-bearing, not cosmetic.** The source data contains ROE readings of 4,744% (BEL) and 893% (INDIGO), caused by equity figures on a different scale from profit. Uncapped, those two companies compress every other company into a narrow band at the bottom of the profitability axis.
- **A missing metric scores a neutral 50, not 0.** Scoring it 0 would punish a company twice for a documented edge case: a turnaround company already has a null PAT CAGR because its base year was a loss.
- **Debt Free companies take the best available coverage reading** rather than being treated as missing, so the leverage component rewards them.
- **A sector-relative variant** normalises within `broad_sector`, answering "good for a bank" rather than "good".

Peer percentile ranking

SQL-style `PERCENT_RANK`, $(\text{rank} - 1) / (n - 1)$, on a 0 to 100 scale, computed for 10 metrics within each of the 11 peer groups. Debt to equity is inverted so less debt ranks higher. A company with a missing metric is excluded from the ranking population rather than ranked last, so a peer group is not penalised for incomplete source data. 550 rows; the 37 companies in no peer group return the message `No peer group assigned` rather than raising.

Clustering

KMeans, `k=5`, `random_state=42`, on five features: ROE, D/E, revenue CAGR, FCF CAGR and operating margin. Missing values are imputed from the sector median, then features are winsorised at P5/P95, then `StandardScaler` is applied.

Why winsorisation was added to the clustering pipeline

The specified pipeline is `impute, scale, cluster`. Run exactly that way it produced three degenerate clusters: one holding BEL and HAL alone with a mean ROE of 4,280%, one holding CIPLA alone, and 58 of the 92 companies dumped together. `StandardScaler` normalises spread but does not stop one company with bad source data from defining an axis. Capping at P5/P95 produced clusters of 13, 26, 28, 15 and 10, each with a recognisable financial character.

5. API design

Read-only REST surface over the analytics layer.

Router	Endpoints	Notes
health.py	1	Status, row counts for 10 tables, uptime, version
companies.py	9	List, profile, P&L, BS, cash flow, ratios, tearsheet PDF, peer compare, documents
screener.py	2	Threshold and preset screening, preset listing
sectors.py	2	Sector list with medians, companies in a sector
peers.py	2	Group list, group detail with percentile ranks
valuation.py	1	Historical multiples, marked SIMULATED
portfolio.py	3	Percentile stats, clusters, outliers

Design decisions

- **The universe is cached per process** with `lru_cache` in `dependencies.py`. The composite score is cross-sectional, so it cannot be computed for one company on request; building it per call would put a full index recomputation behind every endpoint. A warm screener call is 13 ms, a cold one about a second.
- **Connections are per request, not shared.** SQLite connections are not safe to share across threads, so each query opens and closes its own.
- **NaN becomes null, not the string "NaN".** `frame_to_records()` handles the conversion once so no endpoint emits invalid JSON.
- **CORS is fully open and there is no authentication.** Correct for an internal read-only tool; it must not be exposed publicly in this form.
- **Timing middleware** logs method, path, status and elapsed time, and stamps `X-Response-Time-ms` on every response.

Status code contract

Code	Condition
200	Success. Also returned when a company has no peer group, with an explanatory message rather than an error
400	Threshold outside its documented range, unknown preset name, or a year not in YYYY-MM form
404	Unknown ticker, sector or peer group; or no tearsheet exists for that company
422	FastAPI query validation, for example <code>limit=0</code>
500	Unhandled error, returned as JSON rather than HTML

The OpenAPI 3.1 specification is exported to `docs/openapi.json` and is the authoritative contract.

6. Test strategy and performance

311 tests, zero warnings, 20 of 20 acceptance gates.

Suite	Covers
tests/etl/	Year normalisation and the validator rules
tests/dq/	The 14 data quality rules, one test per rule
tests/kpi/	Ratio and CAGR formulas with plain numbers, including every edge case branch
tests/analytics/	Period parsing, peer percentiles, valuation flags
tests/screener/	Filter logic, both business rules, composite score bounds and ordering
tests/nlp/	Parser regex including the deliberate rejections, all 24 rules, confidence bands, fallback behaviour
tests/reports/	Distress and deleveraging flags, PDF page count and size
tests/dashboard/	All 8 screens loaded headlessly via AppTest, plus regression tickers
tests/api/	69 tests: every endpoint, every error path, and API-versus-Excel agreement
tests/test_makefile.py	Static Makefile integrity, since make is not installed here

Testing approach

- **Pure functions are tested with numbers, not fixtures.** The ratio and CAGR modules take scalars, so their edge cases are asserted directly.
- **Screens are tested by loading them.** Streamlit's AppTest runs each page headlessly, so a broken import or an unguarded None fails the suite rather than the demo.
- **Regression tickers are named in the tests.** PNB broke the profile screen because a bank reports no operating profit; it is now a permanent test case alongside ADANIGREEN (short history) and ITC (control).
- **Cross-consumer agreement is asserted.** A test compares API preset output against the Excel workbook, so the two paths cannot silently diverge.
- **Known data defects are pinned, not ignored.** The orphan ticker rule accepts the two documented orphans and fails on any new one.

Measured performance

Measure	Result	Target
10 concurrent screener API calls	0.150 s total	under 10 s
API endpoint latency	under 20 ms median	not specified
Company Profile screen	p95 0.16 s, worst 0.99 s	under 3 s
Indexed query improvement	45% to 69% faster	n/a
Tearsheet batch, 91 PDFs	several minutes	offline batch

Twelve indexes were added on `company_id` and `year`. Honest caveat: at these table sizes SQLite was never the bottleneck, and the absolute saving is tens of microseconds. The real cost sits in matplotlib rendering during the PDF batch, which is offline by design.

7. Data quality and known limitations

Every defect found in the supplied data, and how the platform handles it.

Valuation data is simulated

market_cap and stock_prices are simulated datasets. Every P/E, P/B, EV/EBITDA, dividend yield and market cap figure derives from them, and every surface that shows one labels it SIMULATED. The consequence goes beyond labelling: simulation destroyed the correlations real screening thresholds assume. P/E and P/B are uncorrelated here ($r = -0.11$) where in real markets they move together, which is why the Value Pick preset returns 2 companies instead of a usable shortlist.

Defect	Impact	Handling
companies.xlsx is truncated at 92 rows, stopping part way through the alphabet	Eight tickers appear in the statements with no master record; two of them (ULTRACEMCO, UNIONBANK) reach financial_ratios	Accepted and pinned by a DQ rule that fails on any new orphan. They are excluded from search but included in aggregates
ROE and ROCE scale artifacts	BEL reports 4,744% ROE, INDIGO 893%. Equity and profit are not on the same scale for these companies	Ratio engine retained as correct; winsorisation prevents them distorting scores and clusters; all cases itemised in ratio_edge_cases.log
119 duplicate company-year rows	Join fan-outs; PNB has 60 rows for 12 years	Preserved in source per the Sprint 2 decision; every analytics consumer de-duplicates first
SBIN has no financial_ratios rows	The designated Public Sector Banks benchmark is missing, so that group has no highlighted benchmark	Reported in the workbook and on the peer screen
analysis covers 5 companies, prosandcons covers 16	Neither is usable as an index-wide source	analysis is used to validate the CAGR engine instead; all pros and cons are generated from financial rules
10 broad sectors, not the 11 the documents state	Sector counts read as 11 only because of an Unclassified bucket	Stated plainly wherever a sector count appears
52 annual report URLs are absent	Those years cannot be linked	Rendered with a Report unavailable badge

Analytical caveats worth knowing

- **The distress flag over-reports lenders.** CFO negative with CFF positive is the ordinary signature of loan book growth. 9 of 13 flagged companies are profitable Financials, one with 14,451 Cr of net profit. distress_alerts.csv carries a structurally_normal_for_sector column separating those from the four worth reviewing.
- **Pearson correlation overstates several relationships.** ROE against asset turnover reads +0.96 Pearson and +0.49 Spearman; net profit margin against interest coverage reads +0.79 against +0.09. Where the two disagree the coefficient reflects a few extreme companies, not the index. The heatmap carries a footnote naming the worst pairs.
- **Rule confidence is not probability.** It measures how far a company clears a threshold, nothing more.
- **PRO-13 and CON-13 are relative, not absolute.** They fire only when no absolute rule did. A CON-13 means nothing was found, not that something is wrong.
- **composite_quality_score is latest-year only.** It is cross-sectional, so back-filling it onto historical rows would imply a comparison never made.

8. Build, run and extend

How to reproduce every output and where to add new work.

Full rebuild from source Excel

```
make load # Excel to SQLite
make ratios # financial_ratios + capital_allocation.csv
make indexes # 12 SQLite indexes
make screener # screener, peer ranks, radar charts
make valuation # valuation summary and flags
make nlp # parse analysis text, generate pros/cons
make cashflow # cash flow intelligence
make cluster # KMeans archetypes and elbow plot
make stats # heatmap, outliers, percentiles
make tearsheets # 91 company PDFs, 11 sector, portfolio
make acceptance # analyst guide + 20 gates + checklist
make test # 311 tests

# or the whole chain in dependency order
make all
```

If make is unavailable, every target is a single python `-m src...` call and can be run directly. `tests/test_makefile.py` verifies statically that every target resolves.

Serving

```
streamlit run src/dashboard/app.py # port 8501
uvicorn src.api.main:app --port 8000 # port 8000, /docs
```

Both run simultaneously without conflict; this is verified as part of the Day 43 integration check.

Extension points

To add	Where	Watch out for
A new KPI	A pure function in <code>src/analytics/ratios.py</code> or <code>cashflow_kpis.py</code> , then a column in <code>ratio_engine.py</code> and <code>schema.sql</code>	Guard every input for None; three call sites may use it
A screener metric	A block in <code>screener_config.yaml</code> plus the column in <code>universe.py</code>	Decide whether a missing value should pass or fail
A dashboard screen	A file in <code>src/dashboard/pages/</code>	Add the <code>sys.path</code> bootstrap; read data only through <code>utils/db.py</code> so it stays cached
An API endpoint	A route in the matching router under <code>src/api/routers/</code>	Use <code>frame_to_records</code> so NaN becomes null; add a test in <code>tests/api/</code>
A pros/cons rule	<code>src/nlp/pros_cons_generator.py</code> , with any new history series in <code>features.py</code>	Give it a confidence scale; absolute rules must not be confused with the relative fallbacks

Conventions

- Three-space indentation throughout. This predates the author and is applied consistently; black is deliberately not run because it would reformat 72 of 92 files to no benefit. Ruff passes clean.
- All monetary values are INR Crore.
- `company_id` is upper case and trimmed at load time.
- A calculation that cannot be performed returns None. It does not raise, and it does not return 0.
- Generated artifacts are committed, because the deliverables tracker expects them present. `output/final_deliverables/` is the one exception, being a duplicate copy.

Where to look next

`docs/analyst_guide.pdf` is the user manual. `docs/Sprint1..6_retrospective` covers what was decided and why, sprint by sprint. `output/ratio_edge_cases.log` itemises every ratio anomaly with a category. `output/perf_notes.md` holds the performance measurements. `docs/acceptance_checklist.pdf` records the 20 gates with their evidence.